

湖北大学 The paper of course exam

Name: Big Data Analysis and Applications

Content: Course Report (Class assignments、thesis、report)

Institution: Teacher: Li Jie

Grade and major:

Teacher's comments	Total score	Grading teacher

Digital Lifestyle Analysis: High-Risk Screening, Digital Dependence Prediction, and Lifestyle Profile Clustering

1. Analysis Goals and Tasks

This report follows the workflow required by the course: define the analysis goal, introduce the dataset, clean and transform the data, conduct statistical visualization, train and evaluate machine learning models, and finally interpret the results. The main purpose is not to claim that every model is perfect. The purpose is to make each step explainable and supported by data evidence.

1.1 Background

Digital lifestyle data combine device use, notifications, social media time, sleep, study,

and **activity records**. These variables provide a practical setting for studying whether behavior patterns are associated with **High Risk labels**, **digital dependence scores**, and **exploratory lifestyle profiles**.

1.2 Key Questions

Overall research question: Under an **increasingly digital lifestyle**, which **daily behaviors** should be treated as **priority warning signs** of **digital dependence**, and **how** can these **signals** **guide personal self-management** and **public digital-wellbeing support**?

Three specific questions are used to answer it:

1. **Which behavior patterns** most clearly distinguish **High Risk** and **No Risk** samples? (Classification)
2. Can these **patterns** support **High Risk screening** and **predict digital dependence more** reliably than **productivity**? (Regression)
3. Can **lifestyle profiles** translate these findings into different support strategies? (Clustering)

1.3 Analysis Tasks

The report answers the overall question in **three** connected steps. **EDA** first identifies the strongest warning patterns. **Classification** and **regression** then test whether these patterns can support early screening and dependence estimation. **Clustering** finally turns the patterns into different lifestyle profiles for targeted guidance. The logic therefore moves from **finding warning signals**, to **testing their predictive value**, and then to **designing practical responses**.

2. Dataset and Task Support

2.1 Dataset Source

The dataset is the 2025 Digital Lifestyle Benchmark Dataset, published by Tarek Masryo on **Kaggle** and available through **Hugging Face**. It contains **3,500 fully synthetic records and 24 fields** in CSV format.

2.2 Raw Data Preview

Table1 is used to confirm that the report starts from the raw CSV rather than from a pre-summarized result table.

Dataset Viewer

Split (1)
train · 3.5k rows

Search this dataset

id	age	gender	region	income_level	education_level	daily_role	device_hours_per_d
1	40	Female	Asia	High	High School	Part-time/Shift	
2	27	Male	Africa	Lower-Mid	Master	Full-time Employee	
3	31	Male	North America	Lower-Mid	Bachelor	Full-time Employee	

Downloads last month: 57

Number of rows: 3,500 Total file size: 614 KB

Table1 Raw Dataset Preview

	A	B	C	D	E	F	G	H	I	J	K	L
	id	age	gender	region	income_level	education_level	daily_role	device_hours_per_d	phone_unlock	notifications_per_d	social_media_min	study_min
1	1	40	Female	Asia	High	High School	Part-time/Shift	3.5400	45	561	98	34
2	2	27	Male	Africa	Lower-Mid	Master	Full-time Employee	5.6500	100	393	174	102
3	3	31	Male	North America	Lower-Mid	Bachelor	Full-time Employee	8.8700	181	231	595	140
4	4	41	Female	Middle East	Low	Master	Caregiver/Home	4.0500	94	268	18	121
5	5	26	Female	Europe	Lower-Mid	Bachelor	Full-time Employee	13.0700	199	91	147	60
6	6	37	Female	Middle East	Upper-Mid	Bachelor	Caregiver/Home	4.5900	73	198	9	85
7	7	19	Female	Europe	Lower-Mid	High School	Unemployed/Looking	4.9500	119	553	61	188
8	8	33	Female	Europe	Upper-Mid	Master	Full-time Employee	8.5500	82	184	48	155
9	9	43	Female	South America	Lower-Mid	PhD	Full-time Employee	8.0500	155	309	16	116
10	10	41	Female	Asia	Low	High School	Full-time Employee	2.3300	38	110	249	155
11	11	47	Female	South America	Lower-Mid	High School	Caregiver/Home	7.8900	129	212	31	37
12	12	13	Female	Asia	Low	High School	Student	4.5900	139	474	39	78
13	13	26	Male	Africa	Lower-Mid	Master	Full-time Employee	8.5500	110	424	82	0
14	14	21	Female	Europe	Low	Bachelor	Student	3.9900	72	211	178	102
15	15	24	Female	Africa	Lower-Mid	Master	Full-time Employee	8.9100	92	185	161	201
16	16	24	Female	Africa	Low	Bachelor	Full-time Employee	9.7400	180	1195	240	88
17	17	24	Female	Africa	Upper-Mid	Bachelor	Student	5	114	155	19	102
18	18	25	Male	North America	Lower-Mid	Master	Full-time Employee	5.2900	119	226	125	96
19	19	19	Male	Africa	Lower-Mid	Bachelor	Full-time Employee	6.8900	184	263	41	119
20	20	15	Male	Europe	Low	High School	Student	12.6600	203	703	176	128

Table1 Raw Dataset Preview

The raw data already fits the report design. In Table1, each record contains digital behavior, lifestyle information, and outcome-style fields in one row. This means the dataset is not only used for one model. It can support **High Risk screening**, **score prediction**, and **lifestyle clustering** in the same workflow.

2.3 Field Structure

Table2 lists every original field with its type and role, which makes the later leakage-control decision traceable.

	A	B	C	D	E
	field_name	dtype	non_null_count	example_value	field_meaning
1	id	int64	3500	1	Unique record identifier.
2	age	int64	3500	40	User age.
3	gender	str	3500	Female	User gender category.
4	region	str	3500	Asia	User region.
5	income_level	str	3500	High	Income-level category.
6	education_level	str	3500	High School	Education-level category.
7	daily_role	str	3500	Part-time/Shift	Main daily role or occupation status.
8	device_hours_per_day	float64	3500	3.5400	Daily digital-device use time in hours.
9	phone_unlocks	int64	3500	45	Number of phone unlocks per day.
10	notifications_per_day	int64	3500	561	Number of notifications received per day.
11	social_media_mins	int64	3500	98	Daily social-media time in minutes.
12	study_mins	int64	3500	34	Daily study time in minutes.
13	physical_activity_days	float64	3500	7	Physical activity days per week.
14	sleep_hours	float64	3500	9.1238	Daily sleep duration in hours.
15	sleep_quality	float64	3500	3.3536	Sleep-quality score.
16	anxiety_score	float64	3500	9.9267	Synthetic anxiety-related outcome score.
17	depression_score	float64	3500	5	Synthetic depression-related outcome score.
18	stress_level	float64	3500	6.5933	Synthetic stress-level outcome score.
19	happiness_score	float64	3500	8	Synthetic happiness score.
20	focus_score	float64	3500	23	Synthetic focus score.
21	high_risk_flag	int64	3500	0	Classification target: 1 means High Risk and 0 means No Risk.
22	device_type	str	3500	Android	Main device type.

Table2 Dataset Fields

The **field roles** need to be **separated** before modeling. Table2 marks `high_risk_flag` as the classification target, while `digital_dependence_score` and `productivity_score` are outcome-style scores. If these fields are used carelessly as inputs, the model may **learn the answer too directly**. Therefore, this table supports the later **leakage-control** step.

2.4 Task Support

`high_risk_flag` is the **classification** target, `digital_dependence_score` is the main **regression** target, `productivity_score` is used as a weak-prediction **check**, and **behavior/lifestyle variables** are used for **clustering**.

3. Data Cleaning and Feature Engineering

3.1 Data Loading

The loading step verifies the input file shape and column structure. The raw dataset contains exactly 3,500 records and 24 fields, matching the dataset requirement and the later task design.

3.2 Missing and Duplicate Check

Table3 checks **every field** before any modeling step.

	A	B	C
1	field_name	missing_count	missing_rate
2	id	0	0
3	age	0	0
4	gender	0	0
5	region	0	0
6	income_level	0	0
7	education_level	0	0
8	daily_role	0	0
9	device_hours_per_day	0	0
10	phone_unlocks	0	0
11	notifications_per_day	0	0
12	social_media_mins	0	0
13	study_mins	0	0
14	physical_activity_days	0	0
15	sleep_hours	0	0
16	sleep_quality	0	0
17	anxiety_score	0	0
18	depression_score	0	0
19	stress_level	0	0
20	happiness_score	0	0
21	focus_score	0	0
22	high_risk_flag	0	0
23	device_type	0	0
24	productivity_score	0	0
25	digital_dependence_score	0	0

Table3 Missing Value Check

The **missing-value check** makes the preprocessing simpler. Table3 reports **0 missing values** in all 24 fields, so no imputation is needed. This keeps all **3,500 records** in the analysis and avoids adding artificial filling rules before modeling.

Table4 tests whether repeated rows or **duplicated** identifiers should be removed.

	A	B	C
1	check_item	value	result
2	Total samples	3500	Recorded
3	Duplicate rows	0	No deletion needed
4	Duplicate id values	0	No deletion needed
5	Processing decision	Keep all records	Passed

Table4 Duplicate Check

The dataset also **passes** the **duplicate check**. Table4 reports **0 duplicate rows** and **0 duplicate id values**. Therefore, no sample is removed at this step. This makes the later train/test result more reliable because repeated records are not inflating the evaluation.

3.3 Range and Rationality Check

Table5 summarizes the minimum, maximum, and mean values of key numerical fields.

	A	B	C	D	E	F
1	field_name	min	max	mean	reasonable_range	passed
2	age	13	50	28.0786	10 to 90 years	PASS
3	device_hours_per_day	0.2800	17.1600	7.3177	0 to 24 hours/day	PASS
4	phone_unlocks	9	374	147.1074	0 to 500 unlocks/day	PASS
5	notifications_per_day	22	1211	335.1171	0 to 1000 notifications/day	CHECK
6	social_media_mins	0	631	159.3629	0 to 720 minutes/day	PASS
7	study_mins	0	418	108.1166	0 to 720 minutes/day	PASS
8	physical_activity_days	0	7	3.3500	0 to 7 days/week	PASS
9	sleep_hours	3	11.0046	7.2545	0 to 14 hours/day	PASS
10	sleep_quality	1	5	2.7088	0 to 5 score	PASS
11	productivity_score	33	95	65.2993	0 to 100 score	PASS
12	digital_dependence_score	5.6000	89.2000	36.6842	0 to 100 score	PASS

Table5 Range and Rationality Check

The range check suggests that the wide values are still usable. In Table5, device_hours_per_day ranges from 0.28 to 17.16, and notifications_per_day reaches 1,211. These values show large behavior differences, but they are still meaningful in a digital-lifestyle dataset. So the report keeps the records and handles scale differences later.

3.4 Feature Engineering

Table6 records the engineered behavior-intensity variables.

	A	B	C	D	E	F	G	H	I	J
1	device_hours_per_da	phone_unlock	notifications_per_da	social_media_min	study_mir	physical_activity_da	sleep_hours	sleep_quality	social_media_hour	study_hours
2	y	s	y	s		ys			s	
2	3.5400	45	561	98	34	7	9.1238	3.3536	1.6333	0.5667
3	5.6500	100	393	174	102	2	8.8375	2.9081	2.9000	1.7000
4	8.8700	181	231	595	140	1	6.4867	2.8892	9.9167	2.3533
5	4.0500	94	268	15	121	4	7.6005	3.0975	0.3000	2.0167
6	13.0700	199	91	147	60	1	5.1990	2.7861	2.4500	1
7	4.5900	73	198	9	85	0	10.9452	2.7970	0.1500	1.4167
8	4.9500	119	553	61	188	4	8.4223	3.0651	1.0167	3.1333
9	6.5500	82	184	48	155	3	6.7952	2.9523	0.8000	2.5833
10	8.0500	155	309	16	116	4	9.6984	3.0658	0.2667	1.9333
11	2.3300	38	110	249	155	5	9.7227	3.1874	4.1500	2.5833
12	7.8900	129	212	31	37	4	6.9644	3.0747	0.5167	0.6167
13	4.5900	139	474	39	78	7	7.3168	3.3827	0.6500	1.3000
14	8.5500	110	424	82	0	4	5.9178	3.1063	1.9667	0
15	3.9900	72	211	178	102	4	6.0491	3.0781	2.9667	1.7000
16	8.9100	92	185	161	201	1	5.7790	2.7610	2.6833	3.3500
17	9.7400	180	1195	240	88	7	5.2031	3.2826	4	1.4667
18	5	114	155	19	102	3	9.6170	2.9756	0.3167	1.7000
19	5.2900	119	226	125	96	2	6.2678	2.9270	2.0833	1.6000
20	6.8900	184	263	41	119	5	8.2276	3.1637	0.6833	1.9533
21	12.6600	203	703	176	128	6	5.1989	3.2202	2.9333	2.1333

Table6 Engineered Features Preview

Feature engineering makes the behavior variables easier to interpret. Table6 changes raw minutes and counts into intensity features such as notifications_per_device_hour and device_to_sleep_ratio.

3.5 Feature Selection and Leakage Control

Table7 explains which columns are excluded before modeling.

	A
1	drop_column
2	anxiety_score
3	depression_score
4	digital_dependence_score
5	focus_score
6	happiness_score
7	high_risk_flag
8	id
9	productivity_score
10	stress_level
11	

Table7 Feature Selection and Leakage Control

Leakage control protects the **credibility** of the models. Table7 **removes target-like fields** such as `high_risk_flag`, `digital_dependence_score`, `productivity_score`, `anxiety_score`, `depression_score`, `stress_level`, `happiness_score`, and `focus_score` from the High Risk classifier inputs. This means the classifier must rely on **behavior and lifestyle evidence**, not direct outcome information.

3.6 PCA and GMM Extension

Table8 provides the numerical evidence for the PCA extension.

	A	B	C	D	E	F	G	H
1	component	component_index	explained_variance_ratio	cumulative_explained_variance	input_feature_count	explained_variance_percent	cumulative_explained_variance_percent	interpretation_note
2	PC1	1	0.2694	0.2694	15	26.9447	26.9447	PC1 + PC2 explain about 42.41% of the variance, which is useful for visualization only.
3	PC2	2	0.1547	0.4241	15	15.4680	42.4128	
4	PC3	3	0.1366	0.5607	15	13.6621	55.0748	
5	PC4	4	0.1258	0.6865	15	12.5760	68.6508	
6	PC5	5	0.1030	0.7895	15	10.3018	78.9526	
7	PC6	6	0.0759	0.8654	15	7.5881	86.5407	
8	PC7	7	0.0430	0.9084	15	4.2954	90.8361	
9	PC8	8	0.0425	0.9509	15	4.2529	95.0890	
10	PC9	9	0.0274	0.9783	15	2.7374	97.8264	
11	PC10	10	0.0152	0.9935	15	1.5202	99.3466	
12	PC11	11	0.0040	0.9974	15	0.3952	99.7417	
13	PC12	12	0.0017	0.9991	15	0.1660	99.9077	
14	PC13	13	0.0009	1	15	0.0923	100	
15	PC14	14	0.0000	1	15	0.0000	100	
16	PC15	15	0	1	15	0	100	
17								

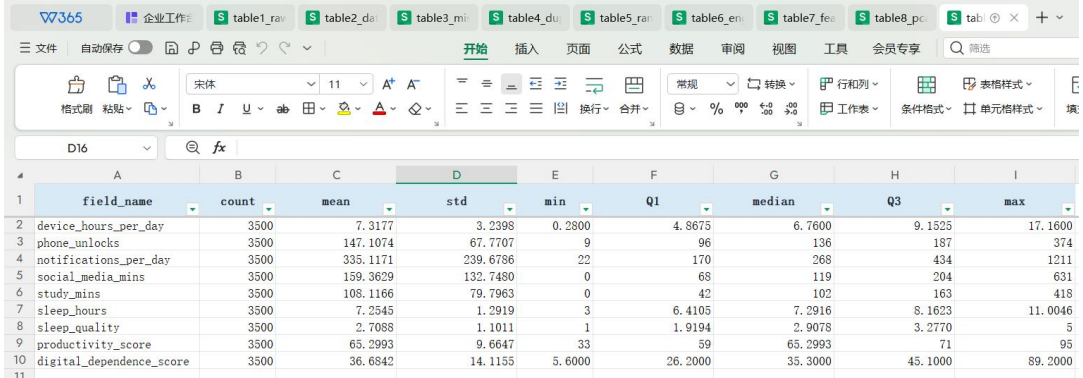
Table8 PCA Explained Variance

PCA is useful, but it cannot **replace** the full feature space. Table8 shows that the first two components explain **42.41%** of the variance. **More than half** of the information is **still outside** the two-dimensional view. Therefore, PCA is used only for **structure understanding**, not as the main input for classification or regression.

4. Statistics and Visualization

4.1 Descriptive Statistics

Table9 gives the **mathematical summary** before visual analysis.



	A	B	C	D	E	F	G	H	I
	field_name	count	mean	std	min	Q1	median	Q3	max
2	device_hours_per_day	3500	7.3177	3.2398	0.2800	4.8675	6.7600	9.1525	17.1600
3	phone_unlocks	3500	147.1074	67.7707	9	96	136	187	374
4	notifications_per_day	3500	335.1171	239.6786	22	170	268	434	1211
5	social_media_mins	3500	159.3629	132.7480	0	68	119	204	631
6	study_mins	3500	108.1166	79.7963	0	42	102	163	418
7	sleep_hours	3500	7.2345	1.2919	3	6.4105	7.2916	8.1623	11.0046
8	sleep_quality	3500	2.7088	1.1011	1	1.9194	2.9078	3.2770	5
9	productivity_score	3500	65.2993	9.6647	33	59	65.2993	71	95
10	digital_dependence_score	3500	36.6842	14.1155	5.6000	26.2000	35.3000	45.1000	89.2000

Table9 Descriptive Statistical Summary

The **descriptive statistics** explain why **scaling is needed** later. In Table9, notifications_per_day and social_media_mins have much wider ranges than bounded variables such as sleep_quality. This makes scaling necessary for **PCA and clustering**, while **tree-based models** can still work with raw feature values.

4.2 Target Distribution

Fig1 is used to check the class structure before model training.

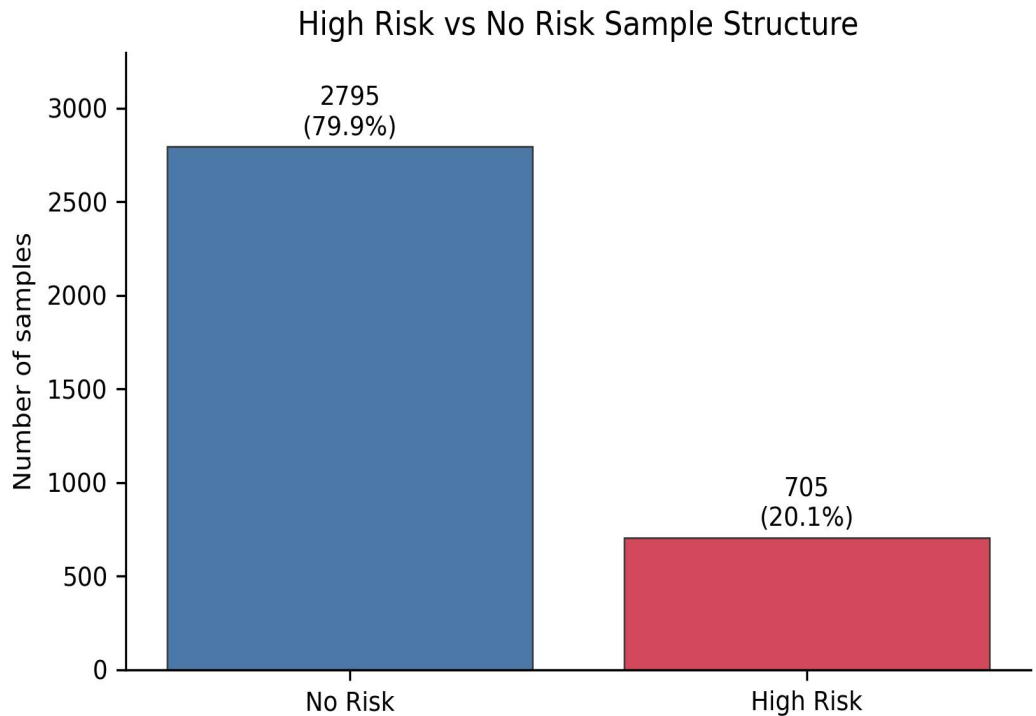


Fig1 High Risk vs No Risk Sample Structure

The first modeling issue is **class imbalance**. As shown in Fig1, No Risk takes up **79.86%** of the samples, while High Risk takes only **20.14%**. If the report only used Accuracy, the model might look good by predicting mostly No Risk. Therefore, the classification section focuses on **Recall, F1, PR-AUC, and the confusion matrix**, because these metrics show whether High Risk samples are actually found.

4.3 Numeric Feature Distribution

Fig2 summarizes the main numerical variables before modeling.

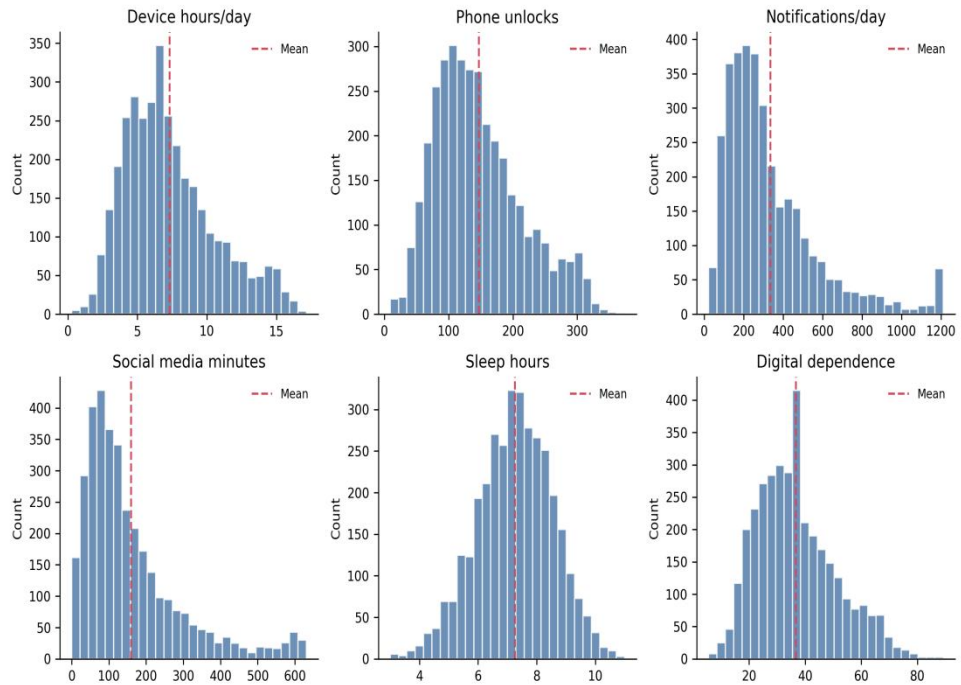


Fig2 Core Digital Lifestyle Feature Distributions

The main issue in Fig2 is the difference in feature scale. Notifications_per_day reaches 1,211, while sleep_hours remains between about 3 and 11 hours. In distance-based methods, variables **with larger ranges** would have too **much influence** if the raw values were used directly. Therefore, the data are **standardized** before **PCA and clustering**, while tree models are less affected because they split variables separately.

4.4 Correlation Structure

Fig3 checks the **correlation structure** among behavior variables, lifestyle variables, and outcomes.

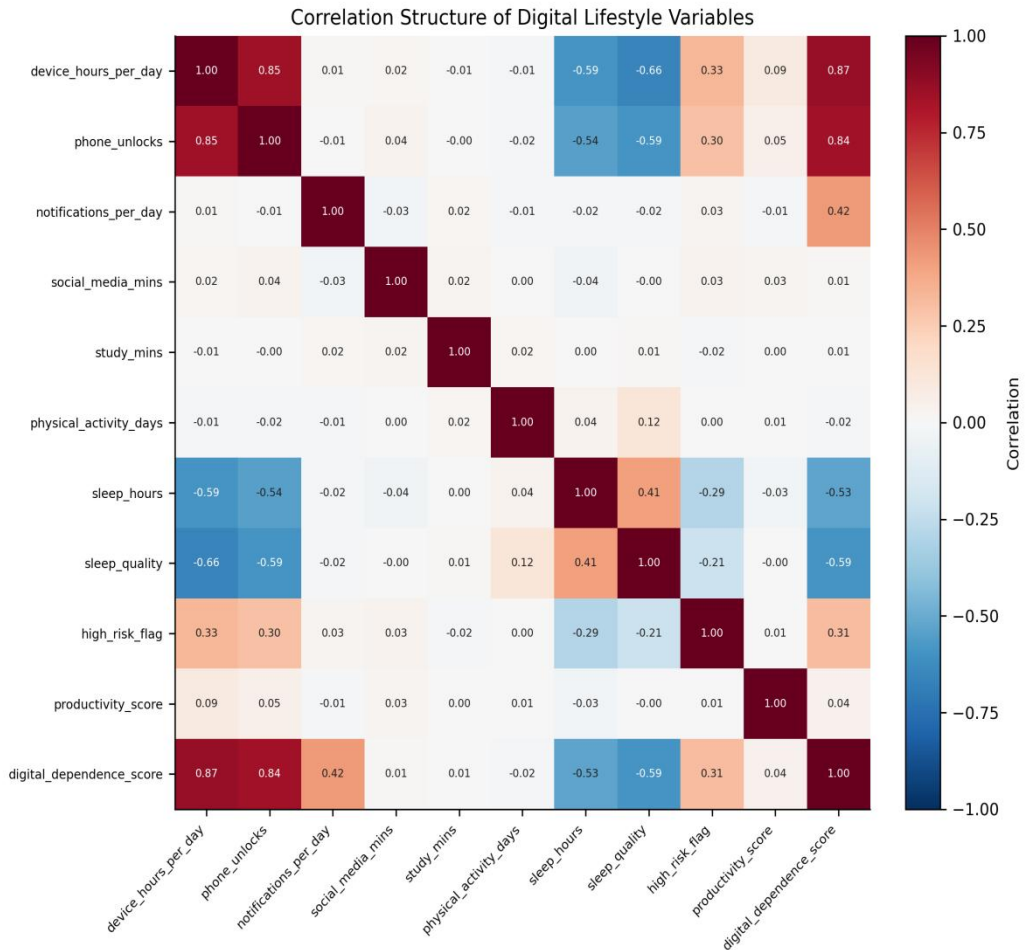


Fig3 Correlation Structure of Digital Lifestyle Variables

Fig3 explains why the two regression targets produce very different results. **digital_dependence_score** has a strong positive relationship with **device_hours_per_day**, with $r=0.87$, and a negative relationship with **sleep_quality**, with $r=-0.59$. In contrast, **productivity_score** has only a weak relationship with **device_hours_per_day**, with $r=0.09$. This pattern suggests that the current features contain strong predictive information for **digital dependence**, but little information for **productivity**. The heatmap only gives **correlation evidence**, so the report needs a more detailed analysis **later**.

4.5 High Risk and No Risk Group Difference

Fig4 compares the **High Risk** and **No Risk** groups using the variables with direct behavioral meaning.

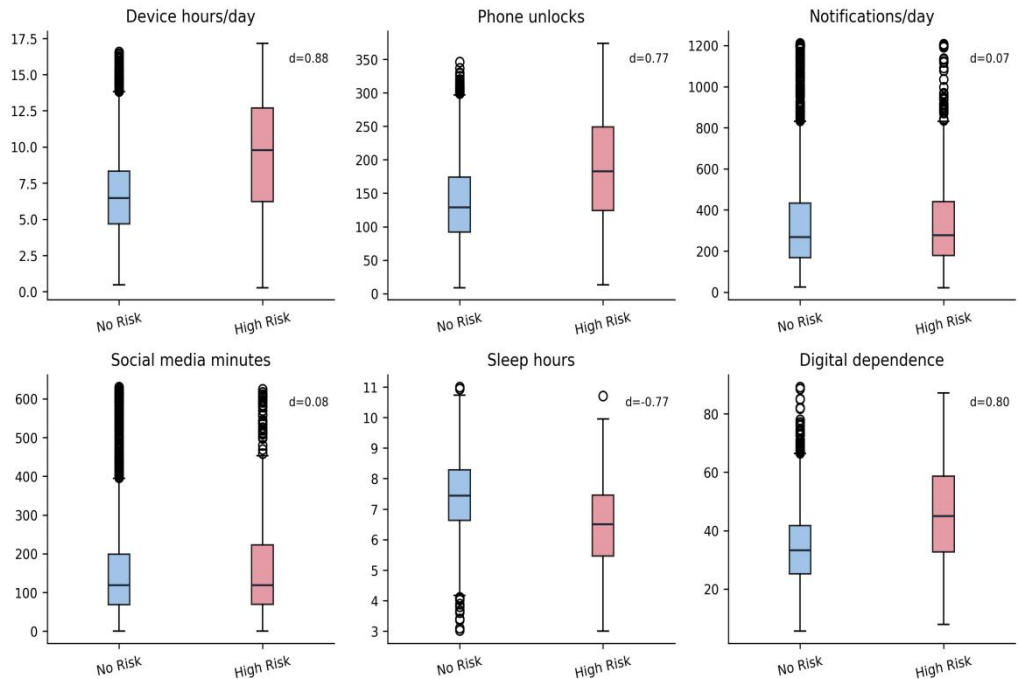


Fig4 Behavioral Difference Between High Risk and No Risk Groups

High Risk is more connected with **long device use** and **weaker sleep balance** than with **notification amount**. In Fig4, the High Risk group has higher device_hours_per_day and higher digital_dependence_score, while sleep_hours is lower. The strongest group difference is device_hours_per_day, but notifications_per_day is weak. Since the two groups still overlap, one single variable cannot separate High Risk from No Risk. This is why the later classifier needs **multiple features** and **threshold tuning**.

5. Model Training and Evaluation

5.1 Experimental Setup

All supervised experiments use **train/test separation**, `random_state=42`, cross-validation or grid-search evidence, and final testing on the **held-out test set**. **Classification** focuses on **Recall**, **F1**, **PR-AUC**, **ROC-AUC**, **Balanced Accuracy**, and the **confusion matrix**. **Regression** focuses on **R2**, **MSE**, **RMSE**, and **MAE**. **Clustering** uses **inertia**, **Silhouette**, **Calinski-Harabasz**, **Davies-Bouldin**, and **profile interpretability**.

Because High Risk represents only **20.14%** of the data, **PR-AUC** is used to compare how well the models **retrieve** and **rank** the **minority class**(our high risk is minority so we used **highest PR-AUC**) across different thresholds. Recall and F1 are then used to choose the final screening threshold.

5.2 High Risk Classification

Fig5 compares the **three candidate classifiers** before threshold adjustment.

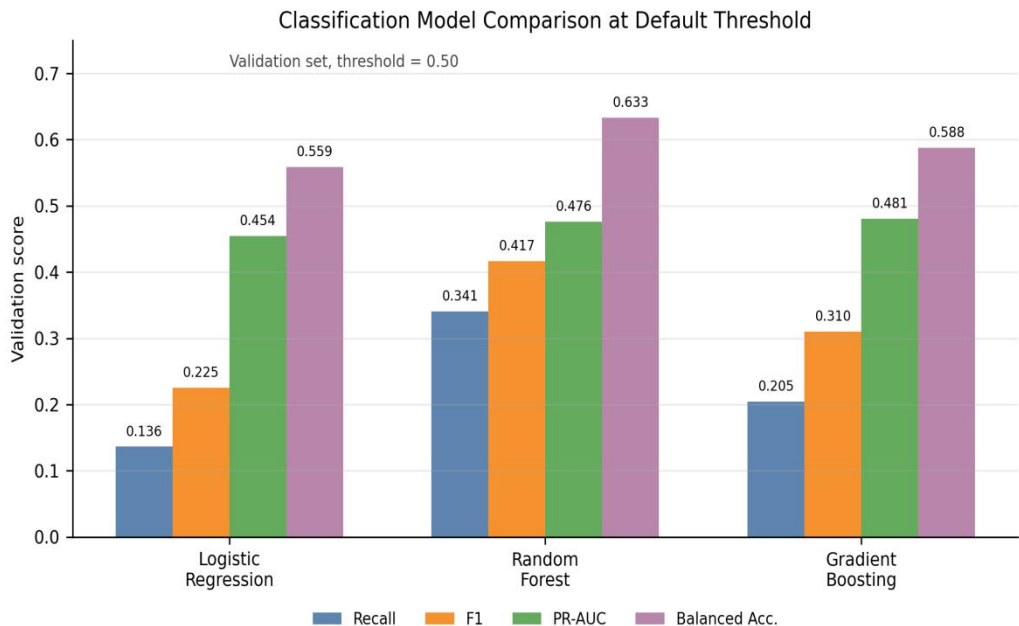


Fig5 Classification Model Comparison at the Default Threshold

The model comparison in Fig5 shows that **model choice** is **not** decided by **one score**. **High Risk** represents only **20.14%** of the samples, so overall **Accuracy** can look **acceptable even** when **many High Risk cases** are **missed**. **PR-AUC** is the area under the **Precision-Recall**

curve. **Precision** asks **how many** warned samples are truly High Risk, while **Recall** asks **how many real** High Risk samples are found. A **higher PR-AUC** therefore **means** that the model can **place more rare** High Risk **cases** near the top of the risk ranking **while controlling false** alarms across different thresholds.

At **threshold=0.50**, **Random Forest** has **better** Recall, F1, and Balanced Accuracy. **However, Gradient Boosting** has the **highest** validation **PR-AUC** at **0.4805**, compared with **0.4759** for Random Forest and **0.4543** for Logistic Regression. Its advantage over Random Forest is only **0.0046**. **Therefore, Gradient Boosting** is **selected** as the slightly **better High Risk ranking** model, while **threshold=0.14** is **chosen** separately in Fig6 to **increase Recall**.

Fig6 explains why the final **classifier** does not use the **default threshold** of 0.50.

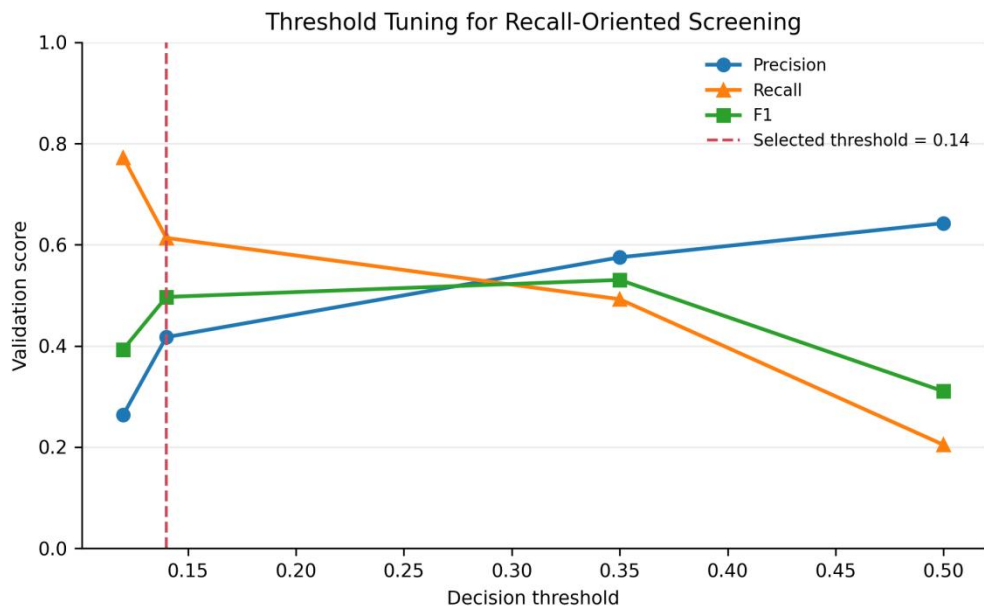


Fig6 Threshold Tuning for Recall-Oriented Screening

The **final threshold** is chosen for **screening rather than** for maximum overall **accuracy**. As **Fig6** shows, **precision** rises as the **threshold** increases, while **recall** falls quickly. Because the **task aims** to **miss fewer** High Risk **samples**, **threshold=0.14** is used. On the **test set**, **Recall** reaches **0.6420**, although the **lower threshold** also creates **more false** alarms. This is a **deliberate screening trade-off**.

Fig7 provides the final **test-set confusion matrix** for **Gradient Boosting at threshold 0.14**.

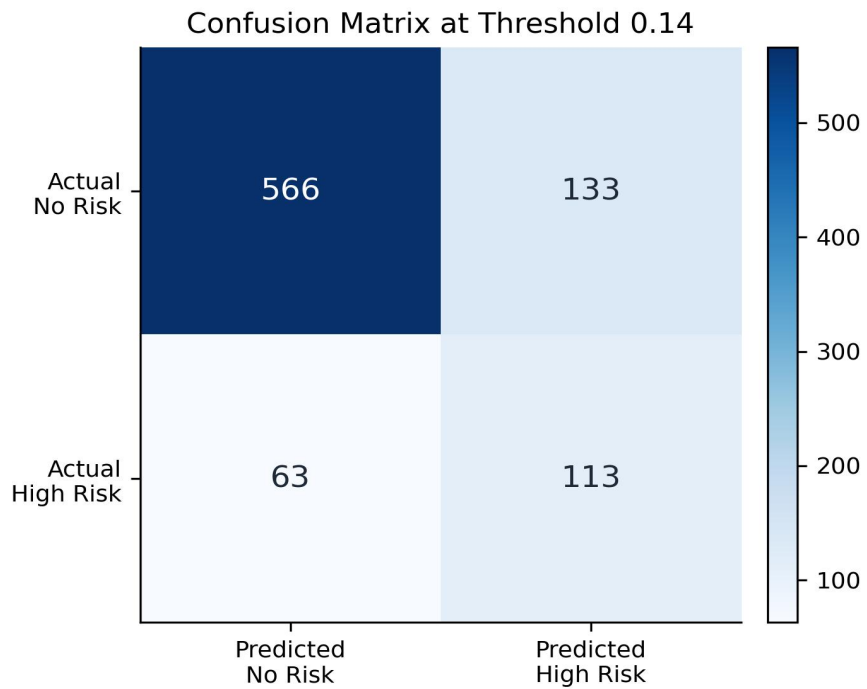


Fig7 Confusion Matrix of the Final High Risk Classifier

The **confusion matrix** makes the screening cost clear. In Fig7, the model finds **113** true High Risk samples, but it still misses **63**. It also creates **133** false alarms. This trade-off is expected because the **threshold was lowered** to find **more High Risk cases**. So the final classifier should be used as a **screening reference**, not as a final personal judgment.

Fig8 evaluates the **High Risk ranking ability** across **thresholds**.

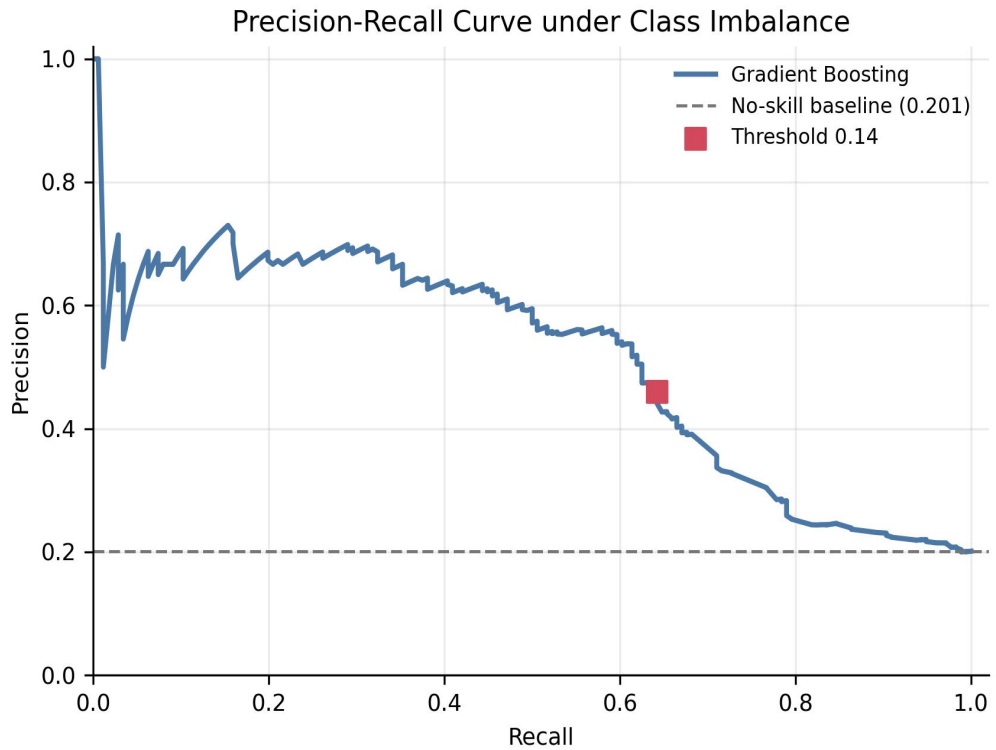


Fig8 Precision-Recall Curve under Class Imbalance

The PR curve shows that the **classifier has ranking value**, but it is **not perfect**. In Fig8, **PR-AUC** is clearly **higher** than the **no-skill baseline**, so the model is **better than random ranking**. However, the curve also shows that **high recall** and **high precision** are **hard to get at the same time**. This supports the **same conclusion** as Fig6 and Fig7: the model is useful for **early screening**, but it still needs human or follow-up judgment.

```
from sklearn.model_selection import StratifiedKFold, GridSearchCV,
train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import precision_recall_fscore_support
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```

from sklearn.preprocessing import OneHotEncoder, StandardScaler

X = engineered.drop(columns=classification_drop)
y = engineered["high_risk_flag"]
num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(exclude="number").columns.tolist()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])

pipe = Pipeline([
    ("preprocess", preprocess),
    ("model", GradientBoostingClassifier(random_state=42)),
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(
    pipe,
    {"model__learning_rate": [0.03, 0.05, 0.1], "model__max_depth": [2,
3]},
    scoring="average_precision",
    cv=cv,
)

grid.fit(X_train, y_train)
proba = grid.predict_proba(X_test)[: , 1]
pred = (proba >= 0.14).astype(int)
print(precision_recall_fscore_support(y_test, pred, average="binary"))

```

Code explanation: The pipeline uses a **stratified split** to **preserve the High Risk ratio** and **GridSearchCV** with **average precision** to **tune Gradient Boosting**. The final **probability scores** are **converted** into screening labels with **threshold=0.14**.

5.3 Digital Dependence and Productivity Regression

Fig9 checks whether the **final digital_dependence_score predictions align** with the **observed scores**.

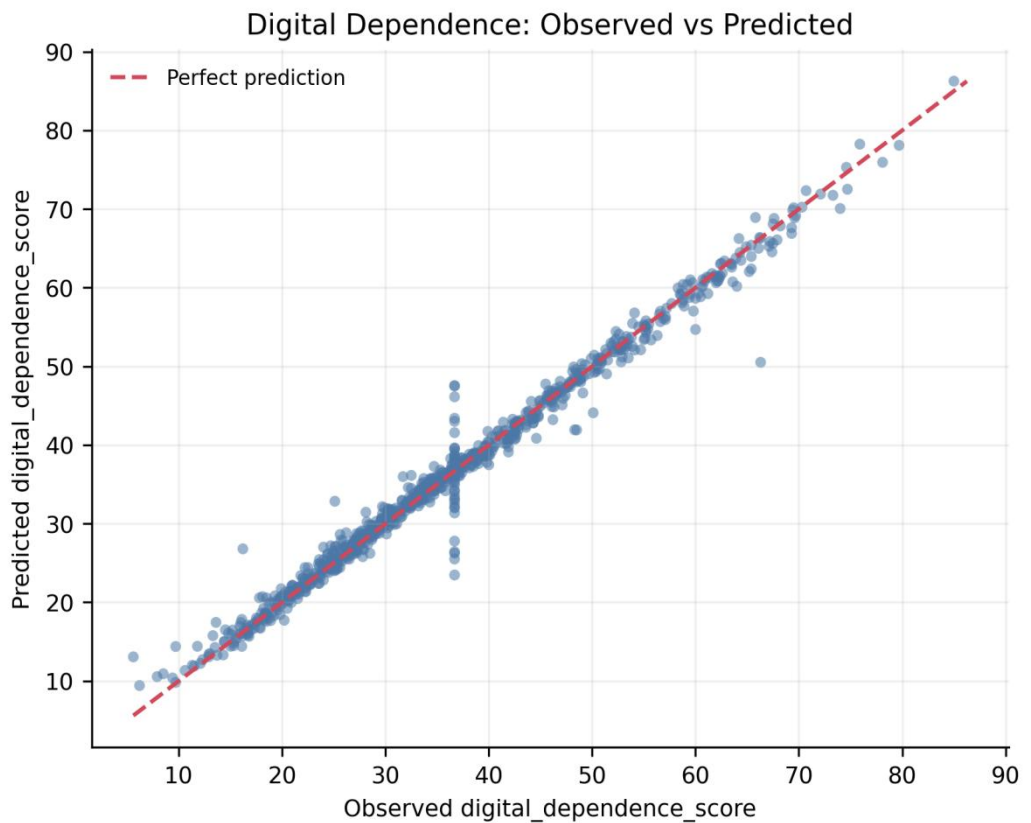


Fig9 Digital Dependence Observed vs Predicted Scores

The **digital-dependence prediction** is the **best result** in the report. In Fig9, most points **stay close** to the diagonal line, and the final R^2 reaches **0.9839**. This means the **current behavior** and **lifestyle variables** describe **digital_dependence_score** very well inside this benchmark dataset. The conclusion is about **prediction**, not **causality**.

Fig10 keeps the **productivity_score** experiment as a **negative-result** check.

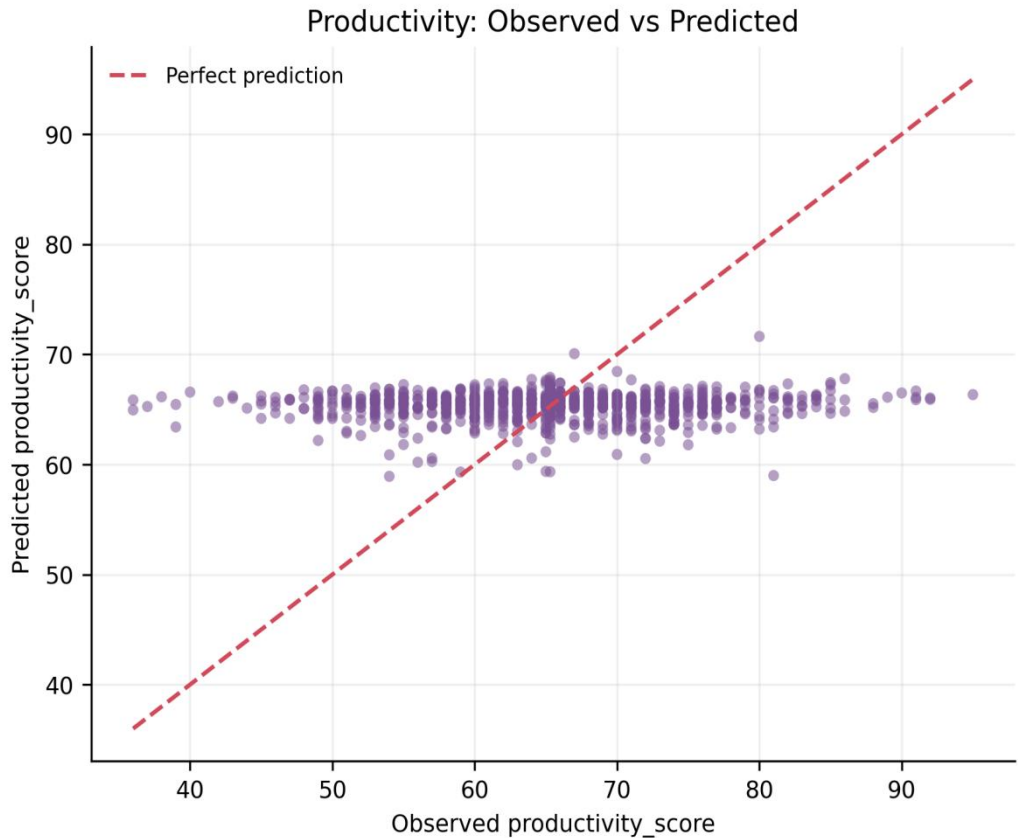


Fig10 Productivity Weak Prediction Observed vs Predicted Scores

The **productivity result** is **weak**, and this weakness is meaningful. In Fig10, the predicted values stay **near the average** instead of following the full spread of actual productivity_score. This means the **model cannot tell high-productivity and low-productivity samples apart** well. With **R^2 below zero**, the **current features** are **not enough** for productivity prediction. This **negative result** shows the **boundary of the dataset**.

Fig11 summarizes the **R2 contrast** between the **two regression targets**.

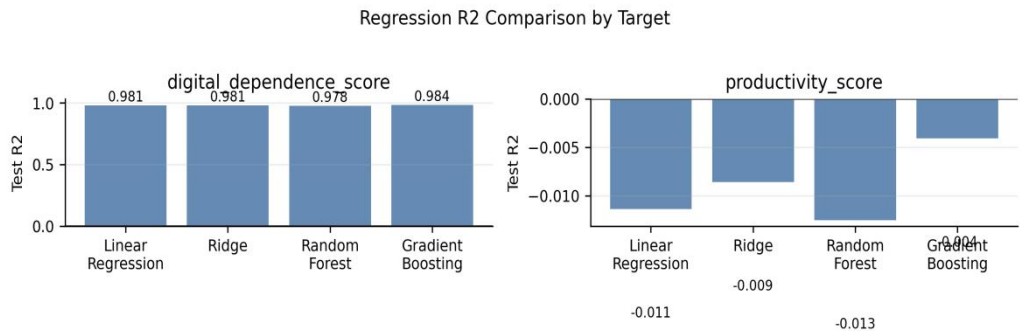


Fig11 Regression R2 Comparison by Target

The contrast in **Fig11** makes the **regression finding clearer**. The **same feature** set works very **well** for **digital_dependence_score**, but it **fails** for **productivity_score**. This is important because it **prevents overclaiming**. The report can say that **digital behavior strongly represents digital dependence**, but it cannot say that the **same behavior** explains **productivity**.

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def regression_metrics(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    return {
        "r2": r2_score(y_true, y_pred),
        "mse": mse,
        "rmse": np.sqrt(mse),
        "mae": mean_absolute_error(y_true, y_pred),
    }

dd_pred = pd.read_csv(PROJECT_ROOT / "results" /
"regression_digital_dependence_predictions.csv")
```

```
prod_pred = pd.read_csv(PROJECT_ROOT / "results" /
"regression_productivity_predictions.csv")

print(regression_metrics(dd_pred["actual"], dd_pred["predicted"]))
print(regression_metrics(prod_pred["actual"], prod_pred["predicted"]))
```

Code explanation: The **same function** calculates **R²**, **MSE**, **RMSE**, and **MAE** for **both targets**, so the **strong digital-dependence** result and **weak productivity** result are **compared** under the **same evaluation rule**.

5.4 Digital Lifestyle Clustering

Fig12 combines **inertia** and **silhouette** evidence to explain the **clustering choice**.

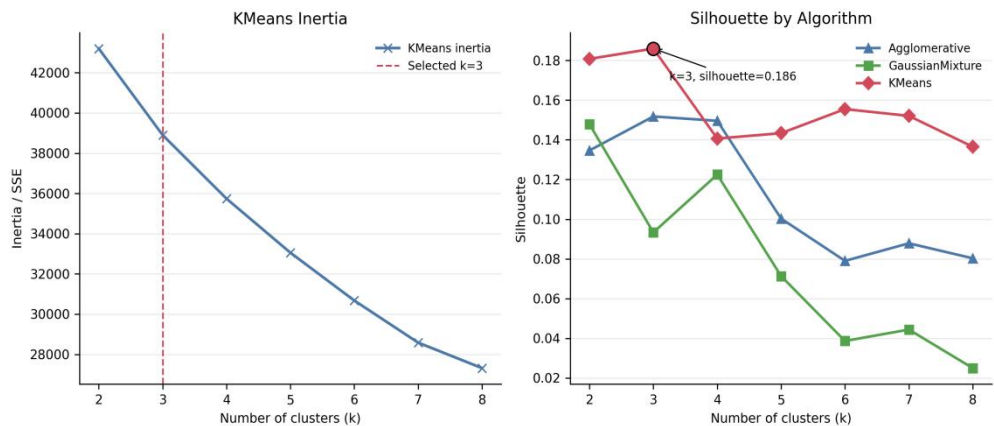


Fig12 K Selection by Inertia and Silhouette

The **clustering result** should be **read carefully**. In **Fig12**, the **inertia** curve does not show a **sharp elbow**, and the **best KMeans silhouette** at **k=3** is only **0.1860**. This means the clusters are **not strongly separated**. Therefore, **k=3** is used because **it gives readable profiles**, not because the data naturally form three clear groups.

Fig13 summarizes the **final KMeans k=3** lifestyle profiles.

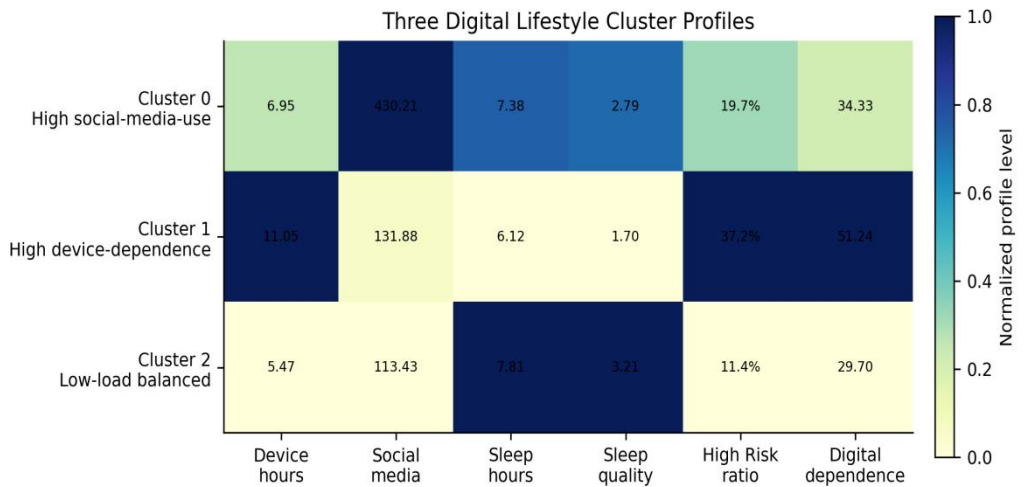


Fig13 Three Digital Lifestyle Cluster Profiles

The **three clusters** have different **behavior focuses**. In **Fig13**, **Cluster 0** has the highest social_media_mins (430.2) but only moderate device_hours_per_day (6.95), so it is mainly a **High social-media-use profile**. **Cluster 1** has the highest device_hours_per_day (11.05), the lowest sleep_quality (1.70), and the highest High Risk ratio (37.2%), so it is the **strongest High device-dependence profile**. **Cluster 2** has the lowest device_hours_per_day (5.47) and the lowest High Risk ratio (11.4%), so it is the **Low-load balanced profile**. However, Silhouette=0.1860 means these are **gradual lifestyle patterns**.

Fig14 provides the **PCA evidence** used to discuss **dimensionality reduction**.

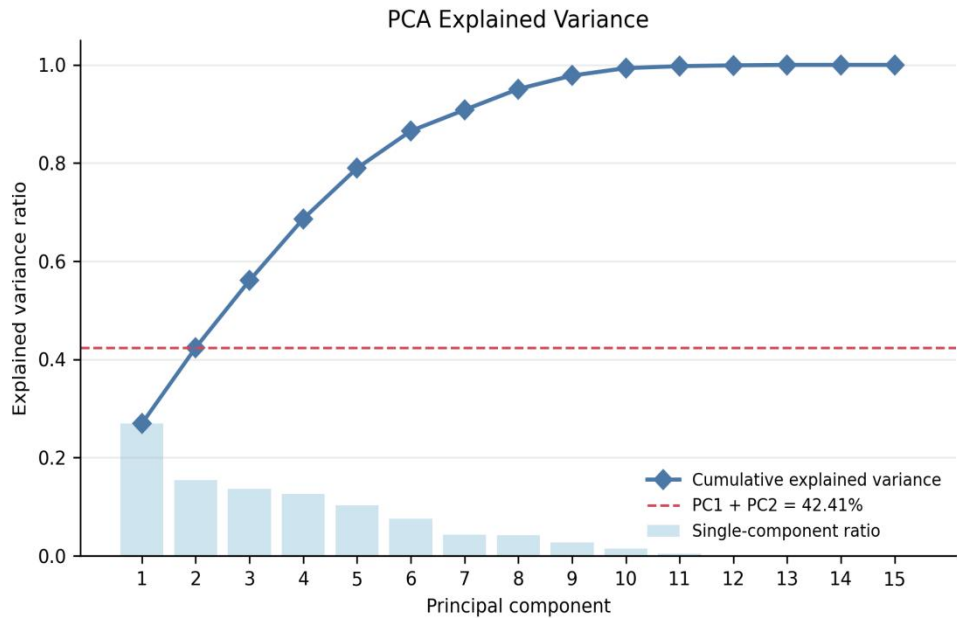


Fig14 PCA Explained Variance for Dimensionality Reduction

PCA helps with **understanding**, but it also **shows** its own **limit**. In **Fig14**, the **first two** principal components explain **42.41%** of the variance, so a **large part** of the information is still **hidden outside** the two-dimensional view. This means PCA can **support visual explanation**, but it should **not replace** the **original feature** space. The final models should still be evaluated with their **own metrics**.

```
from sklearn.cluster import KMeans
from sklearn.cluster import AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

X_cluster = engineered[clustering_features]
X_scaled = StandardScaler().fit_transform(X_cluster)
```



```
scores = []
for k in range(2, 9):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km_labels = km.fit_predict(X_scaled)
    agg = AgglomerativeClustering(n_clusters=k)
    agg_labels = agg.fit_predict(X_scaled)
    gmm = GaussianMixture(n_components=k, random_state=42)
    gmm_labels = gmm.fit_predict(X_scaled)
    scores.extend([
        {"algorithm": "KMeans", "k": k, "silhouette":
silhouette_score(X_scaled, km_labels)},
        {"algorithm": "Agglomerative", "k": k, "silhouette":
silhouette_score(X_scaled, agg_labels)},
        {"algorithm": "GaussianMixture", "k": k, "silhouette":
silhouette_score(X_scaled, gmm_labels)},
    ])

print(pd.DataFrame(scores))
```

Code explanation: The **clustering features** are **standardized** first, and **KMeans**, **AgglomerativeClustering**, and **GaussianMixture** are **compared** for k=2–8. The **silhouette** results support using **KMeans k=3** only as an exploratory **profile solution**.

5.5 Integrated Result Analysis

The overall analysis leads to **one clear conclusion: prolonged device use and weak sleep balance** are the **first behavior patterns** that should be **monitored** when assessing **digital dependence**, while **notification volume** is **not a reliable warning sign**.

The **Gradient Boosting classification** model turns **behavior** and **lifestyle variables** into a **High Risk screening score**. With **threshold=0.14**, **Recall** reaches **0.6420**, so the model can **prioritize possible High Risk cases** for follow-up, but it cannot make final personal decisions.

The **Gradient Boosting regression** model **predicts** `digital_dependence_score` **well**, with $R^2=0.9839$ and $MAE=0.9982$. However, the **same features fail to predict** `productivity_score`, with $R^2=-0.0041$, showing that the **dataset** is **useful** for **dependence-related analysis** rather than **general performance judgment**.

The **KMeans Clustering** model with **k=3** further **organizes** the samples into **three** readable **lifestyle profiles**, although **Silhouette=0.1860** means that these profiles are **not strict** natural groups.

Therefore, the **three models form one practical process**: the **Gradient Boosting classifier** identifies **people who may need further attention**, the **Gradient Boosting regression** model provides a **reference estimate** of **digital dependence**, and the **KMeans Clustering profiles** support **different guidance**—**social-media time management** for Cluster 0, **device-use and sleep-balance support** for Cluster 1, and **balanced-habit maintenance** for Cluster 2. This process turns the **model results** into **targeted follow-up and lifestyle guidance**.

6. Findings and Reflection

6.1 Main Findings

The **main finding** is that the **three models** support **different** but **connected uses**.

The **Gradient Boosting classifier** at **threshold=0.14** identifies **64.20% of true High Risk samples**, including **113** true positives, but it also produces **133** false positives. **Therefore**, it is **more suitable** for **prioritizing follow-up** than for **making final** personal judgments.

The **Gradient Boosting regression** model **predicts digital_dependence_score** accurately, with **$R^2=0.9839$** and **$MAE=0.9982$** , while **productivity_score** remains **unpredictable**, with **$R^2=-0.0041$** .

KMeans with k=3 produces **three broad lifestyle profiles**, although **Silhouette=0.1860** shows that their boundaries are weak.

These **results** lead to **practical guidance** at **three levels**.

At the **individual level**, people should **pay more attention** to **prolonged device use**, **long social-media sessions**, and **weak sleep balance**.

At the **school or community level**, **voluntary screening** can be used to **decide** who should **receive a follow-up questionnaire**: **Cluster 0** can receive **social-media time-management** guidance, **Cluster 1** can receive **stronger device-use** and **sleep-balance support**, and **Cluster 2** can focus on **maintaining balanced habits**.

At the **public level**, **education** and **health authorities** can **strengthen digital-wellbeing** and **sleep-health education**, while **platforms** and **device providers** can offer **clearer screen-time summaries** and **nighttime reminders**.

The **practical value** of this project is not only **analyze the data**, but also to **use data** to **decide who** may **need support**, **what behavior** should be **monitored**, and **which type** of **guidance** may be **more suitable**.

6.2 Limitations

The **limitations** show what the **report** should **not overclaim**.

First, the dataset is a benchmark **educational dataset**, so the very **high digital-dependence result** may partly come from the **dataset structure that means its man-made**.

Second, the **classifier** still **misses** 63 High Risk samples and produces 133 false alarms, **even** after lowering the threshold.

Third, **productivity_score** may depend on **factors not included here**, such as motivation, task context, or learning environment.

Fourth, the **clustering boundary** is **weak because of the** the low silhouette score.

Therefore, these **results** are **useful** for course-level analysis, but stronger real-world claims would need **external validation**.

6.3 Personal Reflection

Big Data Analysis and Applications has **helped me** build a **complete understanding** of **data analysis**. Professor's **careful requirements** and **feedback on analysis logic, visualization, model comparison, and conclusion writing** encouraged me to explain not only what the **numerical results** are, but also **why they matter** and **where the evidence**.

The **course** has also provided a **useful foundation** for my **future study** of **machine learning and deep learning**, especially in **data preprocessing, feature engineering, model evaluation, and responsible interpretation**. I now understand that **only score alone is not enough**. The **metric** must **match the problem**, **weak results** should **not** be **hidden**, and **practical conclusions** must **remain**.

Organizing the **project** on **GitHub** was my **own extension** of the **reproducibility idea**. It made the **data, code, figures, and final results** easier to **review** and **manage**. I believe the **knowledge** and **analytical habits** developed through **Big Data Analysis and Applications** will **continue to help me** in **later courses, competitions, ccf-c paper writing, research, and practical data-analysis projects**.

Appendix A Complete Runnable Code

A.1 Environment and Dependencies

The experiment was completed with a **CPU-based Python environment**. The main dependencies are pandas, NumPy, scikit-learn, Matplotlib, openpyxl, python-docx, and joblib. The environment can be **installed with: `pip install -r requirements.txt`**.

To support reproducibility, the complete project—including preprocessing scripts, model results, figures, and final report files—is available in the GitHub repository:

<https://github.com/hansu650/Big-Data-Homework/tree/main>

A.2 Complete Code

```
"""Complete runnable code source for Appendix A.
```

```
Run from the project directory:
```

```
python appendix_A_complete_code.py
```

```
Or run from the repository root:
```

```
python <project_dir>/appendix_A_complete_code.py
```

```
This file rebuilds the final report evidence files without changing the  
validated core CSV metrics. It loads the raw dataset, checks data quality,  
creates engineered features, documents leakage control, regenerates the  
final  
figures, exports screenshot Excel tables, and summarizes the validated  
results.
```

```
"""
```

```
from __future__ import annotations
```

```
import runpy
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
PROJECT_ROOT = Path(__file__).resolve().parent
```

```
RAW_DATA_PATH = PROJECT_ROOT / "data" / "raw" /  
"digital_lifestyle_benchmark_2025.csv"  
PROCESSED_DATA_PATH = PROJECT_ROOT / "data" / "processed" /  
"digital_lifestyle_benchmark_2025_processed.csv"  
RESULTS_DIR = PROJECT_ROOT / "results"  
FIGURE_DIR = PROJECT_ROOT / "figures" / "final_report"  
SCREENSHOT_TABLE_DIR = PROJECT_ROOT / "screenshot_tables"  
RANDOM_STATE = 42  
  
def load_raw_data() -> pd.DataFrame:  
    df = pd.read_csv(RAW_DATA_PATH)  
    print("Raw dataset shape:", df.shape)  
    print("Raw columns:", df.columns.tolist())  
    print(df.head())  
    return df  
  
def check_missing_duplicates_and_ranges(df: pd.DataFrame) -> None:  
    key_numeric = [  
        "age",  
        "device_hours_per_day",  
        "phone_unlocks",  
        "notifications_per_day",  
        "social_media_mins",  
        "study_mins",  
        "sleep_hours",  
        "sleep_quality",  
        "productivity_score",  
        "digital_dependence_score",  
    ]  
    print("\nMissing values:")  
    print(df.isna().sum())  
    print("\nDuplicate rows:", df.duplicated().sum())  
    print("Duplicate id:", df["id"].duplicated().sum())  
    print("\nRange summary:")  
    print(df[key_numeric].agg(["min", "max", "mean"]).T)  
  
def add_engineered_features(df: pd.DataFrame) -> pd.DataFrame:  
    engineered = df.copy()
```

```
eps = 1e-6
engineered["social_media_hours"] = engineered["social_media_mins"] / 60
engineered["study_hours"] = engineered["study_mins"] / 60
engineered["notifications_per_device_hour"] = (
    engineered["notifications_per_day"] /
engineered["device_hours_per_day"].clip(lower=eps)
)
engineered["unlocks_per_device_hour"] = (
    engineered["phone_unlocks"] /
engineered["device_hours_per_day"].clip(lower=eps)
)
engineered["device_to_sleep_ratio"] = (
    engineered["device_hours_per_day"] /
engineered["sleep_hours"].clip(lower=eps)
)
engineered["activity_sleep_interaction"] = (
    engineered["physical_activity_days"] *
engineered["sleep_hours"]
)
engineered["social_to_study_ratio"] =
engineered["social_media_mins"] / (
    engineered["study_mins"] + 1
)
engineered.to_csv(PROCESSED_DATA_PATH, index=False)
print("\nProcessed dataset saved:", PROCESSED_DATA_PATH)
return engineered

def define_feature_sets(engineered: pd.DataFrame) -> dict[str,
list[str]]:
    outcome_cols = [
        "anxiety_score",
        "depression_score",
        "stress_level",
        "happiness_score",
        "focus_score",
        "high_risk_flag",
        "productivity_score",
        "digital_dependence_score",
```

```
]
classification_drop = [
    "id",
    "high_risk_flag",
    "anxiety_score",
    "depression_score",
    "stress_level",
    "happiness_score",
    "focus_score",
    "productivity_score",
    "digital_dependence_score",
]

def regression_drop_for(target: str) -> list[str]:
    drop_cols = {"id", target}
    drop_cols.update(outcome_cols)
    if target == "digital_dependence_score":
        drop_cols.add("productivity_score")
    if target == "productivity_score":
        drop_cols.add("digital_dependence_score")
    return sorted(drop_cols)

digital_dependence_drop =
regression_drop_for("digital_dependence_score")
productivity_drop = regression_drop_for("productivity_score")
clustering_features = [
    "device_hours_per_day",
    "phone_unlocks",
    "notifications_per_day",
    "social_media_mins",
    "study_mins",
    "physical_activity_days",
    "sleep_hours",
    "sleep_quality",
    "social_media_hours",
    "study_hours",
    "notifications_per_device_hour",
    "unlocks_per_device_hour",
    "device_to_sleep_ratio",
    "activity_sleep_interaction",
```



```

        "social_to_study_ratio",
    ]
    classification_features = [c for c in engineered.columns if c not in
classification_drop]
    digital_dependence_features = [c for c in engineered.columns if c not
in digital_dependence_drop]
    productivity_features = [c for c in engineered.columns if c not in
productivity_drop]
    print("\nClassification drop columns:", classification_drop)
    print("Classification feature count:",
len(classification_features))
    print("Digital dependence regression feature count:",
len(digital_dependence_features))
    print("Productivity regression feature count:",
len(productivity_features))
    print("Clustering features:", clustering_features)
    return {
        "classification_drop": classification_drop,
        "classification_features": classification_features,
        "digital_dependence_drop": digital_dependence_drop,
        "digital_dependence_features": digital_dependence_features,
        "productivity_drop": productivity_drop,
        "productivity_features": productivity_features,
        "clustering_features": clustering_features,
    }

def summarize_validated_results() -> None:
    classification = pd.read_csv(RESULTS_DIR /
"classification_tuned_metrics.csv")
    regression_dd = pd.read_csv(RESULTS_DIR /
"regression_digital_dependence_metrics.csv")
    regression_prod = pd.read_csv(RESULTS_DIR /
"regression_productivity_metrics.csv")
    clustering = pd.read_csv(RESULTS_DIR /
"clustering_kmeans_scores.csv")

    final_cls = classification[
        (classification["dataset"] == "test")
        & (classification["model"] == "gradient_boosting")

```

```

        & (classification["threshold"].round(2) == 0.14)
    ].iloc[0]
    final_dd = regression_dd[regression_dd["model"] ==
"gradient_boosting"].iloc[0]
    final_prod = regression_prod[regression_prod["model"] ==
"gradient_boosting"].iloc[0]
    final_cluster = clustering[clustering["k"] == 3].iloc[0]

summary = pd.DataFrame(
    [
        {
            "task": "classification_high_risk",
            "main_result": "Gradient Boosting threshold=0.14",
            "metric_1": f"Recall={final_cls['recall']:.4f}",
            "metric_2": f"F1={final_cls['f1']:.4f}",
            "metric_3": f"PR-AUC={final_cls['pr_auc']:.4f}",
        },
        {
            "task": "regression_digital_dependence",
            "main_result": "Gradient Boosting",
            "metric_1": f"R2={final_dd['r2']:.4f}",
            "metric_2": f"MSE={final_dd['mse']:.4f}",
            "metric_3": f"MAE={final_dd['mae']:.4f}",
        },
        {
            "task": "regression_productivity",
            "main_result": "weak prediction / negative result",
            "metric_1": f"R2={final_prod['r2']:.4f}",
            "metric_2": "",
            "metric_3": "",
        },
        {
            "task": "clustering_lifestyle_profiles",
            "main_result": "KMeans k=3",
            "metric_1":
f"Silhouette={final_cluster['silhouette']:.4f}",
            "metric_2": "",
            "metric_3": "",
        },
    ]

```

```
)  
output = RESULTS_DIR / "final_workflow_report_summary.csv"  
summary.to_csv(output, index=False)  
print("\nValidated result summary saved:", output)  
  
def regenerate_figures_and_screenshot_tables() -> None:  
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)  
    SCREENSHOT_TABLE_DIR.mkdir(parents=True, exist_ok=True)  
    runpy.run_path(str(PROJECT_ROOT / "scripts" /  
"generate_final_report_figures.py"), run_name="__main__")  
    runpy.run_path(str(PROJECT_ROOT / "scripts" /  
"export_screenshot_tables.py"), run_name="__main__")  
  
def main() -> None:  
    print("Random state:", RANDOM_STATE)  
    df = load_raw_data()  
    check_missing_duplicates_and_ranges(df)  
    engineered = add_engineered_features(df)  
    define_feature_sets(engineered)  
    summarize_validated_results()  
    regenerate_figures_and_screenshot_tables()  
    print("\nComplete workflow evidence generation finished.")  
  
if __name__ == "__main__":  
    main()
```